DEVELOPER INTERVIEW KIT

Document Code	AFRI-DIK-01
Roles Covered	Front-End Developer · Back-End Developer
Format	Virtual / Remote Video Interview
Duration	~45–60 minutes
Stack Focus	React · TypeScript · Node.js / NestJS · PostgreSQL · Git/GitHub
Audience	Interviewers & Hiring Panel

> **How to use:** Ask the shared questions for every candidate, then ask **4–5 role-specific** questions. Use follow-ups only when you need more signal. Early-career candidates with strong projects are welcome — weigh **practical ability and judgement** over years of experience.

1. BEFORE THE CALL (REMOTE CHECKLIST)

ITEM	DONE
Confirm role (FE / BE), time zone, and meeting link	☐
Read CV, cover letter, and one GitHub/portfolio project they highlighted	☐
Stable internet, quiet space, camera on; have a backup (hotspot / WhatsApp)	☐
Candidate can screen-share (GitHub, deployed app, or code)	☐
Open **Afrivate-Developer-Interview-Scorecard.docx** — score during/right after the call	☐
Ask them to join 2–3 minutes early to test audio/video	☐

Opening (2 min): Introduce AfriVate briefly, confirm the role, explain the agenda (background → project deep-dive → technical → remote work → their questions), and say you will take notes.

2. SHARED AGENDA (~50 MIN)

BLOCK	TIME	PURPOSE
Warm-up & motivation	5 min	Fit, clarity, communication
Portfolio / project deep-dive	12–15 min	Ownership, real contribution, problem-solving
Role-specific technical	15–20 min	Stack depth matched to the job post
Remote work & collaboration	5–8 min	Reliability for flexible remote work
Candidate questions + close	5 min	Curiosity, mutual fit

3. SHARED QUESTIONS (BOTH ROLES)

Warm-up

1. Walk me through your background and what drew you to AfriVate / this role.

Listen for: clear story; interest in product/impact — not only "I need a job".

2. What does "good code" mean to you in a small, fast-moving team?

Listen for: readability, pragmatism, shipping, feedback — matches AfriVate values.

Project deep-dive (use their application project)

3. Pick the project you are most proud of. What did *you* personally build?

Listen for: "I" vs vague "we"; concrete contributions.

4. What was the hardest technical problem, and how did you solve it?

Listen for: debugging process, trade-offs, learning — not just tool names.

5. If you rebuilt it today, what would you change?

Listen for: self-awareness, growth, humility.

Remote work & ownership

6. How do you stay reliable when working remotely with flexible hours?

Listen for: async updates, deadlines, escalation, Portal/ClickUp-style habits.

7. Tell me about a time you got stuck. How did you unblock yourself, and when did you ask for help?

Listen for: early escalation, documentation, ownership — not silent struggle for days.

8. How do you handle code review feedback you disagree with?

Listen for: respect, evidence-based discussion, willingness to learn.

Close

9. What questions do you have for us?

Strong signal: product, stack, how work is assigned, FE/BE collaboration, growth.

4. FRONT-END DEVELOPER — TECHNICAL QUESTIONS

Aligned to: React, TypeScript, HTML/CSS, hooks, state, forms, routing, REST APIs, accessibility, Git/GitHub. Nice-to-haves (TanStack Query, Tailwind, Supabase, tests) are bonus, not required. Ask **4–5** questions. Prefer discussion + screen-share over trivia.

Core (pick 4)

FE-1. Components & hooks — In React, when do you use `useState` vs lifting state up vs a shared store (Context / Zustand / Redux)?

Listen for: local vs shared needs; avoids overusing global state; Context isn't always for server data.

FE-2. API integration — How do you handle loading, errors, auth failure, and empty states when calling a REST API from React?

Listen for: UX for each state; retries/timeouts; token expiry; UI never left hanging.

FE-3. TypeScript — How has TypeScript helped (or slowed) you in a React project? Give a concrete example.

Listen for: typing props/API responses; avoiding any; practical benefit, not buzzwords.

FE-4. Responsive & accessibility — How do you make a UI work on mobile and stay usable for keyboard / screen-reader users?

Listen for: semantic HTML, labels, focus, contrast, responsive layout — even if basics only.

FE-5. Git collaboration — Describe your branch → PR → review flow. How do you handle merge conflicts?

Listen for: small PRs, clear descriptions, calm conflict resolution.

Optional follow-ups

- **FE-6.** Client state vs server state (e.g. why TanStack Query / caching API data).
- **FE-7.** One performance issue you've fixed (re-renders, large lists, images, bundle size).
- **FE-8.** Screen-share: open a component they wrote and walk through structure and decisions.

Mini scenario (optional, 5 min): "Design a login form in React that talks to an API. What states and edge cases do you handle?"
Listen for: validation, loading/disabled submit, wrong password, network error, redirect after success, careful token storage.

5. BACK-END DEVELOPER — TECHNICAL QUESTIONS

Aligned to: TypeScript, Node (Express/Fastify/NestJS), PostgreSQL, REST APIs, auth/RBAC, validation, security, migrations, Git. Nice-to-haves (Prisma, Redis, queues, Docker) are bonus. Ask **4–5** questions.

Core (pick 4)

BE-1. API design — How do you design a REST endpoint so a React frontend can use it reliably? What belongs in the contract?

Listen for: status codes, consistent error shape, pagination/filtering, docs, auth headers.

BE-2. PostgreSQL — How do you model related data (e.g. users and roles) and keep it consistent? When do you use transactions?

Listen for: relations/FKs, constraints, migrations; transactions for multi-step writes.

BE-3. Auth & authorisation — Difference between authentication and authorisation. How would you protect an admin-only route?

Listen for: identity vs permissions; JWT/session awareness; RBAC; never trust the client alone.

BE-4. Security — Name 2–3 ways an API gets abused and how you prevent them.

Listen for: validation, parameterised queries, hashed passwords, rate limits, secrets in env.

BE-5. Debugging & ownership — A production endpoint suddenly returns 500s. What do you check first?

Listen for: logs, recent deploys, DB errors, reproducible steps — calm structure.

Optional follow-ups

- **BE-6.** Indexes: when you'd add one and what can go wrong if you add too many.
- **BE-7.** Background jobs / webhooks: one real use case they've built or would design.
- **BE-8.** Screen-share: walk through an API route or schema they wrote (auth + validation).

Mini scenario (optional, 5 min): "Users upload a profile photo. Outline the backend flow: API, storage, permissions, and failure cases."

Listen for: auth before upload, file type/size checks, secure storage, metadata in DB, cleanup on failure.

6. SCORING

> **Use the separate scorecard:** *Afrivate-Developer-Interview-Scorecard.docx* (AFRI-DISC-01).
>
> That form has fixed 1–4 anchors, FE/BE observable checks, hard gates, and forced recommendations. Do not invent informal scores in this kit. Complete one scorecard per candidate during or immediately after the call, before discussing the hire with anyone else.

Advance rule (summary): Advance only if every scored competency is ≥ 3, technical depth is ≥ 3, no hard fails/gates, and Confidence is Medium or High. Full rules live in the DOCX.

7. CANDIDATE INVITE (COPY / PASTE)

Subject: AfriVate interview — [Front-End / Back-End] Developer

Hi [Name],

Thank you for applying. We would like to invite you to a **virtual interview** (~45–60 minutes) for the **[Front-End / Back-End] Developer** role.

Please confirm:
— Preferred date/time (with time zone)
— Video link preference if any (we will send a meeting link)

Please join from a quiet place with stable internet and be ready to **screen-share** a project from your portfolio or GitHub.

Looking forward to speaking with you.

AfriVate Technologies Ltd